

Sky Analytics Engine: Architettura Distribuita per l'Analisi Big Data del Traffico Aereo

Sviluppo di una Pipeline Integrata con Spark, NiFi e Airflow

Flavio Simonelli

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Il presente documento descrive in dettaglio la progettazione, lo sviluppo e la validazione sperimentale di Sky Analytics Engine, una piattaforma Big Data concepita per l'analisi distribuita e l'orchestrazione di pipeline analitiche complesse relative al traffico aereo statunitense. SAE integra componenti leader del settore come Apache Airflow per l'orchestrazione, Apache NiFi per l'ingestion asincrona, e Apache Spark per il processamento distribuito su cluster Hadoop. L'architettura proposta si distingue per la sua estrema modularità, implementata attraverso un approccio "polyglot persistence" e una netta separazione tra i piani di controllo e di esecuzione. Il sistema è in grado di gestire dataset massivi, applicando tecniche avanzate di approssimazione algoritmica e modelli di machine learning non supervisionato per estrarre insight operativi con alta efficienza computazionale. I risultati sperimentali confermano la superiorità delle API strutturate di Spark (DataFrame/SQL) rispetto all'approccio a basso livello (RDD) e dimostrano l'efficacia del formato colonnare Parquet nel ridurre drasticamente l'I/O overhead in ambienti distribuiti, sia in implementazioni on-premise che in scenari cloud-native.

Index Terms—Architetture Big Data, Apache Spark, Apache NiFi, Apache Airflow, Orchestrazione di Pipeline, Data Ingestion, Machine Learning, Scalabilità, Polyglot Persistence.

I. INTRODUZIONE

La crescita esponenziale dei dati generati dai sistemi operanti nel settore del trasporto aereo richiede architetture di elaborazione che superino agilmente i limiti imposti dai sistemi monolitici tradizionali. La sfida odierna non riguarda esclusivamente il volume dei dati generati, ma coinvolge anche la velocità con cui questi devono essere acquisiti, trasformati e resi disponibili per le analisi, unita alla varietà dei formati sorgente e delle destinazioni di storage. Sky Analytics Engine è stato concepito e sviluppato per rispondere in modo esaustivo a queste sfide attraverso un'architettura distribuita, altamente modulare e "cloud-ready".

L'obiettivo primario di SAE è fornire una piattaforma che non si limiti ad essere un mero strumento di calcolo, bensì un ecosistema completo in cui ogni fase del ciclo di vita del dato (Ingestion, Storage, Processing, Serving e Monitoring) sia isolata, tracciabile e facilmente estensibile. Il caso d'uso affrontato in questo progetto si focalizza sull'analisi dei dati storici relativi ai ritardi dei voli commerciali negli Stati Uniti nel periodo Gennaio-Aprile 2025, forniti dal Bureau of Transportation Statistics. Questo dominio applicativo offre una

notevole complessità strutturale (come presenza di valori nulli, anomalie statistiche, dipendenze temporali) e rappresenta un banco di prova ideale per l'implementazione di ottimizzazioni algoritmiche avanzate.

Nelle sezioni successive, esploreremo in dettaglio come la modularità implementata a livello di codice sorgente, attraverso l'utilizzo estensivo del pattern GoF Factory e interfacce astratte in Java, si rifletta coerentemente nella modularità dell'infrastruttura di deployment. Questo design permette a SAE di operare con la medesima efficacia e senza alterazioni alla logica di business sia su un ambiente di sviluppo locale containerizzato tramite Docker Compose, sia su un cluster cloud di produzione basato su Amazon Web Services tramite i servizi di AWS EMR ed EC2.

II. ARCHITETTURA DEL SISTEMA E MODULARITÀ

L'architettura di Sky Analytics Engine rappresenta il fulcro innovativo del progetto. È stata concepita seguendo i rigidi principi della "Separation of Concerns" e della "Microservices-oriented Infrastructure". Il sistema rifugge l'approccio monolitico, configurandosi come un'aggregazione di servizi distribuiti coordinati che comunicano esclusivamente attraverso interfacce standardizzate.

A. Decomposizione Funzionale

L'infrastruttura SAE è organizzata in macro-livelli gerarchici indipendenti. Questa netta separazione assicura che il sistema sia "future-proof": è possibile scalare o sostituire il motore di processing (es. passare da Spark a Flink) senza impattare il sistema di orchestrazione o le logiche di visualizzazione.

I livelli principali sono:

- 1) **Orchestration Layer:** Gestisce lo scheduling e lo stato delle esecuzioni.
- 2) **Ingestion Layer:** Si occupa dell'acquisizione sicura e della trasformazione grezza dei dati.
- 3) **Processing Layer:** Esegue il calcolo distribuito e l'addestramento dei modelli ML.
- 4) **Storage Layer:** Implementa la persistenza multi-modello.
- 5) **Monitoring Layer:** Raccoglie la telemetria di sistema.

B. Livello di Orchestrazione

In Sky Analytics Engine, Apache Airflow 3.0 non viene utilizzato per un singolo flusso lineare, ma gestisce una suite di tre DAG specializzati, ciascuno progettato per coprire un aspetto specifico del ciclo di vita del software e della validazione sperimentale.

1) *Disaccoppiamento tramite Apache Livy*: Una caratteristica architetturale di rilievo è l'integrazione di Apache Livy. Nelle architetture legacy, Airflow necessita dell'installazione dei binari client di Spark e Hadoop sui propri nodi worker per lanciare i job tramite `spark-submit`. SAE supera questa limitazione interagendo esclusivamente con l'API REST di Livy. Airflow prepara un payload JSON contenente il path del JAR (precedentemente caricato su HDFS o S3) e gli argomenti di classe, inviandoli a Livy. Questo approccio trasforma di fatto il cluster Spark in una risorsa "Compute-as-a-Service", riducendo il carico computazionale sul nodo orchestratore e migliorando la sicurezza isolando il piano di controllo dal piano dati.

2) *Sincronizzazione Asincrona con NiFi*: L'interazione tra Airflow e NiFi rappresenta una sfida architettonica. SAE risolve questo problema con un pattern di callback asincrono. Airflow avvia il flusso NiFi tramite una chiamata HTTP POST, fornendo un JWT generato a runtime e un identificatore di variabile univoco. Successivamente, Airflow entra in uno stato di *Sensor Polling*, consumando risorse minime. Quando NiFi completa l'ingestion, utilizza il JWT per autenticarsi contro le API REST di Airflow e aggiornare la variabile di stato, sbloccando il DAG e permettendo l'avvio della fase di processing.

3) *Il DAG di Orchestrazione*: Rappresenta il workflow principale di produzione. Questo DAG coordina l'interazione tra NiFi e Spark attraverso un pattern di callback asincrono. Implementa tre modalità operative selezionabili via parametri di trigger:

- **Both**: Triggera NiFi per l'ingestion, attende il completamento tramite un sensore su variabili Airflow, e avvia il processing Spark.
- **Ingest Only**: Si ferma dopo la conversione in Parquet da parte di NiFi, ideale per popolare il data lake.
- **Execution Only**: Salta NiFi e avvia direttamente Spark sui dati già pre-processati, accelerando le iterazioni analitiche.

4) *Benchmark Sequenziale Locale/EC2*: Questo DAG è stato progettato per la fase di valutazione sperimentale sistematica. A differenza del DAG principale, `ec2_benchmark` itera automaticamente su tutte le combinazioni possibili di query e backend. Utilizzando un approccio puramente sequenziale, implementato tramite dipendenze dinamiche generate in un loop Python, garantisce che ogni job Spark non subisca interferenze da esecuzioni parallele, fornendo dati di telemetria puliti e affidabili per il confronto delle performance su infrastrutture standalone o EC2.

5) *Benchmark Cloud-Native EMR*: Sviluppato specificamente per l'ambiente AWS EMR, questo DAG adatta la logica

di benchmarking alle peculiarità del cloud. Non utilizza Livy, ma sfrutta gli operatori nativi Amazon: *EmrAddStepsOperator* per sottoporre i job come "step" del cluster EMR e *EmrStepSensor* per monitorarne il completamento. Questa scelta permette di sfruttare le capacità di log-aggregation di AWS EMR e di gestire job che richiedono risorse massicce, mantenendo una visibilità completa sulla console AWS oltre che su Airflow.

C. Livello di Ingestion

Apache NiFi è la spina dorsale del livello di acquisizione dati. La sua architettura basata sul paradigma Flow-Based Programming garantisce un controllo visivo e transazionale sui flussi di dati, garantendo anche la Data Provenance.

1) *Pipeline di Trasformazione*: Il flusso `SAE_Ingestion_Pipeline` esegue un processo Extract-Load-Transform sofisticato:

- **Estrazione dei Metadati**: Il processore `EvaluateJsonPath` estrae dal payload di Airflow le configurazioni dinamiche (es. bucket S3 di destinazione, URL del dataset).
- **Fetch e Validazione**: `InvokeHTTP` scarica il dataset CSV zippato. Un processore custom calcola l'hash SHA-1 in streaming e lo confronta con la firma attesa, garantendo la non-ripudio del dato.
- **Conversione Colonnare Parquet**: Questo è il passaggio più critico per l'ottimizzazione delle performance a valle. Utilizzando `ConvertRecord` basato su uno schema Avro fortemente tipizzato, NiFi trasforma i dati da formato testuale CSV a formato colonnare Parquet. Questa operazione preserva i tipi nativi (Integer per i tempi, Double per i ritardi) e scarta i record malformati inviandoli a una coda di dead-letter.

2) *Formato dei Dati*: La scelta di Parquet è strategica: la sua struttura a blocchi con dizionari integrati abilita il *predicate pushdown* nel motore Catalyst di Spark. Se una query analizza solo American Airlines, Spark legge dai dischi HDFS/S3 esclusivamente i blocchi Parquet contenenti tale compagnia, riducendo l'I/O di rete di ordini di grandezza rispetto alla scansione lineare di un file CSV.

3) *Pre-processing e Data Quality*:

D. Livello di Processing

Il cuore computazionale di SAE è l'applicazione Spark, sviluppata in Java 11. La modularità del codice sorgente è stata ottenuta applicando rigorosamente i design pattern della programmazione orientata agli oggetti.

1) *Pattern Factory e Repository*: Per astrarre l'accesso ai dati, SAE implementa il pattern Repository. L'interfaccia `FlightRepository` definisce metodi generici come `getFlights()` e `saveResults()`. Tramite la classe `FlightRepositoryFactory`, l'applicazione istanzia a runtime implementazioni specifiche (es. `HdfsFlightRepository`, `PostgresFlightRepository`, `RedisFlightRepository`) basandosi su un file

di configurazione YAML, gestiti attraverso la classe `ApplicationConfig.java`. Questo significa che la logica di business analitica è completamente disaccoppiata dalla tecnologia di storage sottostante.

2) *Astrazione delle Query*: Ogni query estende la classe astratta `BaseQuery`. Questa classe si fa carico del boilerplate code: istanziazione della `SparkSession`, gestione del logging strutturato, iniezione del listener delle metriche e recupero dei dati dal repository. Le classi figlie devono implementare esclusivamente tre metodi: `runQueryRDD`, `runQueryDataFrame` e `runQuerySQL`, focalizzandosi solo sulla trasformazione del dato.

E. Livello di Storage

Un'architettura Big Data moderna non può fare affidamento su un singolo paradigma di database. SAE implementa una strategia "Polyglot Persistence" per ottimizzare i costi e le latenze di accesso in base al caso d'uso:

- **Hadoop HDFS / Amazon S3**: Fungono da *Single Source of Truth*. Conservano i dati raw e pre-processati in Parquet, offrendo throughput elevato per le letture batch di Spark.
- **PostgreSQL**: Utilizzato come *Serving Layer* relazionale. I risultati aggregati delle query vengono salvati qui per essere facilmente interrogati da strumenti di Business Intelligence tradizionali come Grafana.
- **Redis Stack**: Funge da database per le metriche di telemetria emesse in tempo reale da Spark.
- **Apache HBase**: Implementa il paradigma NoSQL a colonne larghe. È impiegato per memorizzare dataset di risultati estremamente granulari, garantendo tempi di accesso nell'ordine dei millisecondi per singole chiavi.

III. DETTAGLI IMPLEMENTATIVI DELLA COMPUTAZIONE

Le quattro query richieste dal progetto sono state implementate in modo da testare diverse astrazioni di Spark e valutare i trade-off tra controllo a basso livello ed engine ottimizzati.

A. Analisi Base e Ottimizzazioni RDD

La *Query 1* richiede il calcolo di statistiche mensili (media, min, max, tasso di cancellazione) per specifiche compagnie. Nell'implementazione RDD, si è evitato l'uso dell'operatore `groupByKey`, noto per causare gravi problemi di Out-Of-Memory a causa del trasferimento incontrollato di dati durante la fase di shuffle. Al suo posto è stato utilizzato `aggregateByKey`. Questo operatore combina i valori parzialmente all'interno di ogni singola partizione del nodo worker prima di trasferirli sulla rete, riducendo l'impronta di memoria e i tempi di rete.

La *Query 2* introduce logiche di *feature engineering* per gestire i dati sparsi, imputando i valori nulli delle cause di ritardo a zero e filtrando i carrier non statisticamente significativi.

B. Approssimazione tramite KLL Sketch

Il calcolo dei percentili su finestre orarie nella *Query 3* rappresenta la sfida computazionale più ardua. Un calcolo esatto richiederebbe l'ordinamento globale di milioni di record, un'operazione che su un cluster distribuito causa enormi colli di bottiglia dovuti allo scambio di dati.

SAE risolve questo problema implementando tecniche di "Quantile Sketching". Attraverso l'integrazione della libreria Apache DataSketches, è stato implementato lo stimatore Karnin-Lang-Liberty. L'algoritmo KLL mantiene in memoria un set compresso e rappresentativo dei dati osservati. La potenza matematica del KLL risiede nella sua proprietà di unione: due sketch generati su nodi worker differenti possono essere sommati nel driver node mantenendo rigorose garanzie matematiche sull'errore di rango massimo (configurato di default a $\sim 1.6\%$). Questo permette di calcolare i percentili orari in un singolo passaggio sui dati, con una complessità spaziale fissa indipendente dalla dimensione del dataset.

Nell'implementazione `DataFrame`, SAE delega questa approssimazione alla funzione nativa `percentile_approx`, la quale utilizza un approccio simile derivato dall'algoritmo Greenwald-Khanna.

C. Machine Learning e Ottimizzazione Modelli

L'obiettivo della *Query 4* è raggruppare le compagnie aeree in base alle loro feature prestazionali.

L'implementazione in `AirlineClustering.java` costruisce una complessa pipeline:

- 1) **Aggregazione Iniziale**: Calcolo delle feature medie per carrier escludendo i voli cancellati dai conteggi di ritardo.
- 2) **Vector Assembler**: Consolidamento delle colonne numeriche in una singola colonna di tipo `Vector`, formato richiesto da MLlib.
- 3) **Standardization**: Applicazione di uno `StandardScaler`. Poiché il k-Means è sensibile alla scala delle feature (basandosi sulla distanza euclidea), è imperativo normalizzare i dati affinché abbiano media 0 e deviazione standard 1, evitando che feature con magnitudo maggiore dominino l'algoritmo.
- 4) **Model Selection**: SAE non richiede di specificare a priori il parametro K , ovvero il numero di cluster. Il sistema esegue un loop iterativo addestrando modelli da $K = 2$ a $K = 8$. Per ogni modello, valuta la qualità del raggruppamento utilizzando il *Silhouette Score* fornito da `ClusteringEvaluator`. Il punteggio di Silhouette misura la coesione intra-cluster rispetto alla separazione inter-cluster. SAE seleziona dinamicamente il K che massimizza questo punteggio, garantendo un clustering statisticamente fondato.

IV. TELEMETRIA DISTRIBUITA

Per garantire l'osservabilità di un sistema distribuito asincrono, SAE implementa un modulo di telemetria altamente personalizzato.

A. *JobTimerListener Custom*

Affidarsi unicamente alla UI nativa di Spark non è sufficiente per analisi storiche automatizzate. SAE implementa la classe `JobTimerListener`, che estende le interfacce native dello `SparkListener`. Questo componente viene agganciato al bus degli eventi di Spark durante l'inizializzazione della sessione.

Il Listener intercetta eventi cruciali come `onJobStart`, `onStageCompleted` e `onTaskEnd`. Registra timestamp millisecondi e metriche relative al volume di dati letti e scritti. Alla fine del ciclo di vita dell'applicazione, aggrega questi dati e, sfruttando la connettività asincrona verso Redis, persiste i record di telemetria. Questo permette di incorporare il "vero" tempo di calcolo sui dati dal tempo fisiologico speso da Spark per negoziare le risorse con YARN o allocare i container.

B. *Visualizzazione in Grafana*

L'ecosistema si chiude con Grafana, configurato per leggere direttamente da Redis (tramite plugin `RediSearch`) e PostgreSQL. Le dashboard sono state progettate per presentare:

- Benchmark prestazionali comparativi.
- Trend temporali dei ritardi suddivisi per vettore.
- Heatmap della distribuzione oraria calcolata sui percentili approssimati.
- La disposizione geometrica dei cluster ML nello spazio delle feature principali.

V. INFRASTRUTTURA DI DEPLOYMENT

L'agilità di SAE è esaltata dalle sue procedure di deployment, strutturate in due macro-scenari.

A. *Configuration Management as Code*

Tutta la configurazione è externalizzata in file YAML e `.env`. L'assenza di stringhe di connessione o path assoluti hardcoded nel codice Java permette di compilare il JAR una sola volta (tramite Maven) e di eseguirlo in contesti completamente diversi.

B. *Ambiente Standalone AWS EC2*

Il primo macro-scenario simula un'infrastruttura di tipo locale o on-premise virtualizzata in cloud. Il deployment si avvale di una suite di istanze Amazon EC2 attestate all'interno della medesima Virtual Private Cloud e collegate in una sottorete privata:

- **Master Node (1x `t3.medium`):** Ospita i servizi di coordinamento primari, tra cui Apache Airflow, Apache NiFi, Apache Livy, il master di Apache Spark, e i database di serving PostgreSQL e Redis. Dispone di 2 vCPU e 4 GB di RAM.
- **Worker Nodes (3x `t3.small`):** Dedicati esclusivamente al calcolo distribuito.
- Ciascun nodo ospita un demone worker di Spark e un datanode HDFS all'interno di container Docker coordinati tramite Docker Compose. Ciascun worker dispone di 2 vCPU e 2 GB di RAM.

In questa configurazione, HDFS funge da file system distribuito primario, mentre la sottomissione dei job Spark avviene in modalità client tramite Apache Livy, garantendo il completo disaccoppiamento logico ed architetturale.

C. *Ambiente Cloud-Native AWS EMR*

Il secondo scenario proietta SAE in una dimensione pienamente elastica ed enterprise-ready, sfruttando il servizio gestito Amazon Elastic MapReduce. L'infrastruttura è configurata come segue:

- **Master Node (1x `m6g.xlarge`):** Coordina il resource manager Hadoop YARN.
- **Core Nodes (3x `m6g.xlarge`):** Eseguono i demoni executor di Spark sotto il controllo di YARN e memorizzano i dati intermedi. Ciascun nodo dispone di 4 vCPU e 16 GB di RAM.

Il vantaggio cardine di questo scenario risiede nell'integrazione nativa dello storage con Amazon S3 mediante il protocollo `s3a://`. Questo abilita un paradigma **Shared-Nothing** puro, in cui le risorse di calcolo possono essere scalate o spente in modo del tutto indipendente dallo storage persistente, azzerando i costi di idle e ottimizzando il total cost of ownership. L'orchestrazione da Airflow avviene in questo caso sfruttando le API native di AWS tramite operatori Spark dedicati.

VI. VALUTAZIONE SPERIMENTALE

Definita l'architettura e implementati i componenti, è stata condotta una fase di validazione sperimentale sistematica per valutare la robustezza, la scalabilità e le performance di Sky Analytics Engine. I test sono stati orchestrati in modo sequenziale tramite il DAG `ec2_benchmark` su EC2 e `emr_benchmark` su EMR, garantendo l'assenza di interferenze computazionali e l'affidabilità della telemetria raccolta.

A. *Analisi Comparativa delle Performance*

Le misurazioni prestazionali raccolte evidenziano trend marcati circa l'efficacia dei diversi backend di Apache Spark e l'impatto dell'infrastruttura di deployment.

1) *Dispersione Statistica dei Backend:* L'osservazione dei box plot (Fig. 1) rivela che le API strutturate presentano mediane prestazionali generalmente inferiori rispetto all'approccio RDD nelle query più complesse (Query 2 e Query 3), mostrando anche una dispersione statistica più contratta nella maggior parte dei casi. La scatola del box plot per DataFrame e SQL risulta estremamente stretta, con code di variabilità minime e una quasi totale assenza di outlier significativi nel caso della Query 1 e 3 su EMR.

Questa notevole stabilità prestazionale è una conseguenza diretta del **Catalyst Optimizer** e del motore d'esecuzione **Tungsten**. Catalyst esegue l'ottimizzazione logica del piano di esecuzione (attraverso regole come il predicate pushdown e la column projection) prima di compilare il bytecode Java, assicurando che la pipeline fisica sia estremamente lineare ed efficiente. Dal canto suo, Tungsten memorizza i record in formato binario off-heap compresso, aggirando del tutto



Figura 1. Dispersione statistica dei tempi di esecuzione su AWS EC2 (sinistra) e AWS EMR (destra) per la Query 1 (in alto), Query 2 (al centro) e Query 3 (in basso). I grafici evidenziano la distribuzione dei tempi per ciascun backend.

l'allocazione di oggetti Java sulla JVM. Ciò riduce a livelli trascurabili l'attività del Garbage Collector, neutralizzando una delle fonti principali di variabilità temporale nei sistemi distribuiti.

Al contrario, le esecuzioni basate su RDD mostrano box più alti e ampi, indicando una marcata dispersione statistica e code di esecuzione lunghe. Non avendo alcuna visibilità sulla struttura dei dati, Spark tratta i record RDD come oggetti Java opachi, costringendo ad operazioni di serializzazione e deserializzazione pesante per ogni operazione di trasformazione o filtraggio. Questo continuo passaggio di oggetti generici carica pesantemente la memoria degli heap della JVM, innescando frequenti e imprevedibili cicli di garbage collection che dilatano casualmente i tempi di esecuzione, producendo la dispersione visibile in Fig. 1.

Inoltre, nella Query 3, la dispersione del backend RDD è accentuata dall'overhead algoritmico del KLL Sketch implementato tramite la libreria esterna Apache DataSketches integrata manualmente. I backend DataFrame e SQL beneficiano invece dell'ottimizzazione nativa della funzione `percentile_approx` basata sull'algoritmo di Greenwald-Khanna incorporato direttamente nelle routine Tungsten, che mantiene un footprint in memoria costante e una dispersione minima.

2) *Impatto dell'Infrastruttura*: Un aspetto cruciale che emerge dall'analisi comparativa è l'impatto dell'infrastruttura sul cosiddetto **Wall Clock Time** (il tempo totale reale percepito dall'utente per completare il job). Come evidenziato dai grafici dettagliati posizionati in Appendice A (Fig. 2), esiste

un overhead fisso di Wall Clock Time che si somma al tempo puro di calcolo degli executor Spark.

Questo overhead della telemetria è causato da fattori estranei al procesamiento dei dati in senso stretto:

- **Inizializzazione e allocazione delle risorse**: Il tempo speso dal Driver per comunicare con il Resource Manager per allocare i container degli executor.
- **Compilazione del DAG**: La pianificazione logica e fisica eseguita da Catalyst a runtime.
- **Negoziante della rete ed I/O**: La serializzazione e la spedizione dei task compilati dal driver ai vari nodi di calcolo.

Questo tempo morto rimane costante (oscillando tra 1.5 e 3 secondi a seconda della query) e incide proporzionalmente di più sulle query rapide (Query 1 e 2). L'infrastruttura gioca un ruolo determinante nel mitigare questo overhead:

- 1) **AWS EMR**: Il cluster gestito minimizza drasticamente il Wall Clock Time overhead grazie all'integrazione nativa ottimizzata con Hadoop YARN e l'uso di vCPU dedicate. EMR accelera l'inizializzazione del job e la comunicazione interna del cluster, dimezzando i tempi di calcolo. La disponibilità di 16 GB di RAM per istanza `m6g.xlarge` previene completamente lo spilling su disco durante lo shuffle, mantenendo le distribuzioni estremamente compatte.
- 2) **AWS EC2 Standalone**: Le istanze containerizzate della famiglia `t3` esauriscono rapidamente i propri crediti CPU sotto sforzo prolungato, subendo un drastico throttling delle prestazioni. Lo spazio in RAM ristretto

costringe Spark a frequenti operazioni di spilling su disco HDFS (configurato su volumi EBS standard), provocando un allungamento dei tempi e un'accentuata dispersione statistica con numerosi outlier.

VII. CONCLUSIONI E SVILUPPI FUTURI

Il progetto Sky Analytics Engine dimostra che l'implementazione rigorosa di pattern architetturali distribuiti e l'adozione di formati dati ottimizzati sono requisiti imprescindibili per gestire la complessità inerente ai Big Data. L'orchestrazione asincrona tra Airflow e NiFi, combinata con la modularità polyglot dell'engine Spark, fornisce un ambiente robusto ed enterprise-ready. L'introduzione di tecniche algoritmiche avanzate, quali i Quantile Sketches e il Model Selection automatizzato tramite Silhouette Score, dota SAE di capacità analitiche di altissimo livello pur mantenendo confinati i costi computazionali.

I potenziali sviluppi futuri della piattaforma includono la transizione da un paradigma puramente batch a un'architettura ibrida Lambda. L'integrazione di Apache Kafka e Spark Structured Streaming permetterebbe a SAE di acquisire eventi di volo in real-time, applicando i modelli ML di clustering precedentemente addestrati per prevedere ritardi anomali "on-the-fly".

RIFERIMENTI BIBLIOGRAFICI

- [1] Apache Spark Foundation, "Apache Spark - A Unified engine for large-scale data analytics," <https://archive.apache.org/dist/spark/docs/3.5.1/>, Consultato: Maggio 2026.
- [2] Apache Livy, "Apache Livy Documentation," <https://livy.apache.org/docs/latest/>, Consultato: Maggio 2026.
- [3] Apache Airflow, "TaskFlow API Tutorial," https://airflow.apache.org/docs/apache-airflow/stable/tutorial_taskflow_api.html, Consultato: Maggio 2026.
- [4] Apache NiFi, "Apache NiFi User Guide," <https://nifi.apache.org/docs/nifi-docs/html/user-guide.html>, Consultato: Maggio 2026.
- [5] Apache Hadoop, "HDFS Architecture Guide," <https://hadoop.apache.org/docs/r3.3.2/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>, Consultato: Maggio 2026.
- [6] Amazon Web Services, "Amazon Simple Storage Service (S3) Documentation," <https://docs.aws.amazon.com/s3/>, Consultato: Maggio 2026.
- [7] Apache Parquet, "Parquet Format Specification," <https://parquet.apache.org/docs/file-format/>, Consultato: Maggio 2026.
- [8] Apache HBase, "Apache HBase Reference Guide," <https://hbase.apache.org/book.html>, Consultato: Maggio 2026.
- [9] PostgreSQL Global Development Group, "PostgreSQL Documentation," <https://www.postgresql.org/docs/15/>, Consultato: Maggio 2026.
- [10] Redis Ltd., "Redis Documentation," <https://redis.io/docs/latest/>, Consultato: Maggio 2026.
- [11] Grafana Labs, "Grafana Official Documentation," <https://grafana.com/docs/>, Consultato: Maggio 2026.

APPENDICE A

DETTAGLIO DEI TEMPI DI ESECUZIONE

In questa appendice vengono riportati in dettaglio i singoli pannelli di telemetria dedicati ai tempi medi di elaborazione Spark estratti da Grafana per i due ambienti di test: AWS EC2 standalone (sinistra) e AWS EMR gestito (destra), per tutte le quattro query eseguite.



Figura 2. Wall Clock Overhead su AWS EC2 (sinistra) e AWS EMR (destra) per la Query 1 (prima riga), Query 2 (seconda riga), Query 3 (terza riga) e Query 4 (quarta riga). I grafici evidenziano i tempi per ciascun backend.

APPENDICE B

DISTRIBUZIONI ORARIE DEI PERCENTILI ESTESE

Per completezza scientifica e accuratezza metodologica del documento, in questa sezione vengono riportati i grafici dettagliati della distribuzione oraria dei percentili di ritardo all'arrivo per tutte e quattro le compagnie aeree target.



Figura 3. Distribuzione oraria completa dei percentili di ritardo all'arrivo per tutte le compagnie aeree target: American Airlines (in alto a sinistra), Delta Air Lines (in alto a destra), United Airlines (in basso a sinistra) e Southwest Airlines (in basso a destra).